

ΜΥΕ041

Διαχείριση Σύνθετων Δεδομένων

Διδάσκων: Ν. Μαμουλής

Εργασία 3 - Indexing and Similarity Search for Dense Multidimensional Vectors

Ομάδα:

Παρασκευόπουλος Παναγιώτης
Μπεζαΐτης Άγγελος

ΑΜ: 2905
ΑΜ: 4432

Εισαγωγή

Η αναζήτηση ομοιότητας σε πολυδιάστατους χώρους αποτελεί ένα κρίσιμο πρόβλημα σε διάφορες εφαρμογές. Καθώς αυξάνονται οι διαστάσεις στα δεδομένα μας διάφορες παραδοσιακές δομές δεδομένων καθίστανται ανεπαρκείς, καθώς ο χρόνος αναζήτησης αυξάνεται με την σειριακή σάρωση ολόκληρων συνόλων δεδομένων.

Σε αυτή την εργασία χρησιμοποιούμε ένα σύνολο από 10.000 αντικείμενα των 10 διαστάσεων το καθένα, καθώς και ένα αρχείο με 10 ερωτήματα των ίδιων διαστάσεων. Στόχος μας ήταν να εντοπίσουμε αντικείμενα είτε εντός μιας συγκεκριμένης απόστασης (Range Queries), είτε με βάση τους k πλησιέστερους γείτονες (k NN Queries). Για αυτό τον λόγο κληθήκαμε να υλοποιήσουμε τρεις μεθόδους για την αξιολόγηση ερωτημάτων ομοιότητας πάνω σε πολυδιάστατα διανύσματα-δεδομένα (Linear Scan/Naive, Pivot-based, και iDistance).

- **Naive (Linear Scan):** Η πρώτη αυτή μέθοδος υπολογίζει την απόσταση κάθε αντικειμένου του dataset από το σημείου ερωτήματος (query point).
- **Pivot-based Search:** Σε αυτή την μέθοδο χρησιμοποιούμε ένα υποσύνολο αντικειμένων ως σημεία αναφοράς και φιλτράρουμε τον χρόνο αποφεύγοντας άμεσα αντικείμενα που είναι μαθηματικά αδύνατο να ανήκουν στα αποτελέσματα.
- **iDistance:** Σε αυτή την μέθοδο “μετατρέπουμε” τον πολυδιάστατο χώρο σε μονοδιάστατο και ομαδοποιούμε τα δεδομένα σε περιοχές με βάση το κοντινότερο pivot. Στην συνέχεια κάνουμε δυαδική αναζήτηση και σαρώνουμε μόνο τα σχετικά διαστήματα.

Μέρος 1: Φόρτωση δεδομένων και υπολογισμός των pivots

Στο πρώτο μέρος της εργασίας μας παίρνουμε τα δεδομένα μας από το αρχείο κειμένου data10K10.txt και τα αποθηκεύουμε σε μία λίστα από tuples. Η επιλογή array εξασφαλίζει το να είναι κάθε αντικείμενο άμεσα προσβάσιμο σε σταθερό χρόνο με βάση το αναγνωριστικό του ID. Για την διαδικασία επιλογής των k pivots βασιστήκαμε στην ευρετική προσέγγιση της μεγιστοποίησης των αποστάσεων μεταξύ τους καλύπτοντας έτσι ομοιόμορφα τον χώρο. Ο αλγόριθμός μας λειτουργεί ως εξής:

- 1) Αρχικά επιλέγουμε το αντικείμενο για σημείο εκκίνησης με ID μηδέν και υπολογίζουμε την Ευκλείδεια απόσταση από αυτό για όλα τα αντικείμενα του συνόλου. Επιλέγουμε στην συνέχεια το αντικείμενο με την μεγαλύτερη απόσταση από το σημείο εκκίνησης ως το πρώτο pivot p_0
- 2) Αρχικοποιούμε έναν δισδιάστατο πίνακα $distances[i][j]$ ο οποίος έχει μέγεθος

(πλήθος αντικειμένων) $\times$ (πλήθος pivots) και όταν βρεθεί το p_0 υπολογίζονται όλες οι αποστάσεις των αντικειμένων από αυτό.

- 3) Στην συνέχεια για να επιλέξουμε κάθε επόμενο pivot ($p_1 - p_{k-1}$) διατρέχουμε ξανά όλα τα αντικείμενα και υπολογίζουμε το άθροισμα των αποστάσεων από τα ήδη προεπιλεγμένα pivots, τα οποία ανακτούμε από τον πίνακα distances που δημιουργήσαμε στο 2ο βήμα και επιλέγουμε ως νέο pivot το αντικείμενο με την μεγαλύτερη αθροιστική απόσταση.
- 4) Τέλος, καταχωρούνται όλες οι αποστάσεις των αντικειμένων από το νέο pivot που επιλέχθηκε και επαναλαμβάνουμε την διαδικασία μέχρι να συμπληρωθεί ο επιθυμητός αριθμός num_pivots επιστρέφοντας μετά την λίστα των IDs των επιλεγμένων pivot και τον πίνακα distances.

Μέρος 2: Μέθοδος iDistance (Δημιουργία Ευρετηρίου)

Στο δεύτερο μέρος της εργασίας μας υλοποιήσαμε την μέθοδο δημιουργίας ευρετηρίου (index) iDistance, η οποία μας επιτρέπει να αναζητούμε εύκολα δεδομένα πολυδιάστατου χώρου μετατρέποντας τα σε μία μόνο διάσταση. Πρακτικά χωρίζουμε τον χώρο σε περιοχές με βάση τα pivots και υπολογίζεται μία μονοδιάστατη τιμή iDistance για κάθε αντικείμενο. Αλγοριθμικά αυτό υλοποιήθηκε ως εξής:

- 1) Αρχικά βρίσκουμε το κοντινότερο pivot. Για κάθε αντικείμενο του συνόλου δεδομένων (o), διατρέχουμε τον πίνακα αποστάσεων distances (που υπολογίσαμε στο Μέρος 1) για να εντοπίσουμε το κοντινότερο σε αυτό pivot (η $\min(o)$). Αυτό σημαίνει ότι κάθε αντικείμενο αντιστοιχίζεται στα partition του πλησιέστερου σημείου αναφοράς. Για βελτιστοποίηση της ταχύτητας, αξιοποιήσαμε τις ενσωματωμένες συναρτήσεις της Python ($\min$ και $.index()$), αποφεύγοντας τα χρονοβόρα for-loops καθώς και για να αποφευχθούν σφάλματα υποδιαστολής κατά την αναζήτηση στα επόμενα μέρη αποθηκεύσαμε το index του κοντινότερου pivot για κάθε αντικείμενο σε έναν βοηθητικό πίνακα `closest_pivot_idx`.
- 2) Ταυτόχρονα με την εύρεση του κοντινότερου pivot, παρακολουθούμε και καταγράφουμε τη μέγιστη απόσταση από κάθε pivot προς οποιοδήποτε αντικείμενο ανήκει στο partition του. Ο πίνακας `maxd_p[i]` διατηρεί την ακτίνα της μεγαλύτερης σφαίρας που περικλείει όλα τα αντικείμενα του pivot i .
- 3) Αφού σαρωθούν όλα τα αντικείμενα, εντοπίζουμε το μέγιστο από όλα τα `maxd_p`. Αυτή η σταθερά είναι κρίσιμη για τη χαρτογράφηση, καθώς εξασφαλίζει ότι τα partitions δεν θα αλληλεπικαλύπτονται στον μονοδιάστατο χώρο.
- 4) Στην συνέχεια υπολογίζουμε την τιμή `iDist` με τον τύπο που μας δόθηκε $iDist = i * maxd + dist(o, p_i)$ όπου i ο δείκτης του κοντινότερου pivot.
- 5) Τέλος, αποθηκεύονται όλα τα ζεύγη (`iDist`, `oid`), όπου `oid` είναι το

αναγνωριστικό του αντικειμένου, σε μία λίστα `idistance_array`, η οποία ταξινομείται σε αύξουσα σειρά. Αυτή η λίστα θα λειτουργήσει ως το κύριο ευρετήριο μας.

Μέρος 3: Ερωτήματα Ομοιότητας Εύρους (Range Similarity Queries)

Στο τρίτο μέρος της εργασίας υλοποιήσαμε τα ερωτήματα ομοιότητας εύρους (Range similarity queries). Στόχος είναι να εντοπίσει όλα τα αντικείμενα του συνόλου δεδομένων που απέχουν από ένα σημείο ερωτήματος (query point) απόσταση μικρότερη ή ίση με ένα δοθέν κατώφλι ϵ (δηλαδή $\text{dist}(q, o) \leq \epsilon$).

Αρχικά για την **Naive Προσέγγιση (Linear Scan)** ο αλγόριθμος σαρώνει σειριακά όλο το σύνολο δεδομένων. Για κάθε αντικείμενο βρίσκει την ευκλείδεια απόσταση και αν η απόσταση είναι μικρότερη ή ίση του ϵ τότε το αντικείμενο προστίθεται στα αποτελέσματα. Αυτό μας εγγυάται ότι θα βρεθούν όλα τα σωστά αποτελέσματα αλλά είναι πολύ χρονοβόρο όταν ασχοληθούμε με τεράστια σύνολα δεδομένων (Big Data).

Στην συνέχεια υλοποιούμε την **Pivot-based προσέγγιση** η οποία εκμεταλλεύεται τον πίνακα `distances` του μέρους 1 για να φιλτράρει υποψήφια αντικείμενα με βάση τον κανόνα της τριγωνικής ανισότητας. Σε πρώτο στάδιο υπολογίζει από κάθε pivot (k υπολογισμοί) την απόσταση με το σημείο ερωτήματος. Μετά για κάθε αντικείμενο o , ελέγχεται η συνθήκη: $|\text{dist}(p_i, o) - \text{dist}(p_i, q)| > \epsilon$, για κάθε pivot. Αν ισχύει η συνθήκη έστω και για ένα pivot το αντικείμενο απορρίπτεται άμεσα (pruning), χωρίς να χρειαστεί να κάνουμε υπολογισμούς απόστασης. Αν περάσει όλους τους ελέγχους τότε θα υπολογιστεί και η πραγματική του απόσταση από το q .

Τέλος για την **iDistance-based Προσέγγιση** υλοποιούμε τον ταχύτερο αλγόριθμο αναζήτησης αξιοποιώντας την μονοδιάστατη δομή του μέρους 2. Σε πρώτο στάδιο κάνουμε pruning στα επίπεδα ολόκληρων περιοχών. Πρακτικά αν η απόσταση του 1 από το pivot μείον την ακτίνα της περιοχής `maxd_p[i]` είναι μεγαλύτερη από το ϵ τότε όλο το partition αγνοείται. Μετά υπολογίζουμε τα όρια αναζήτησης με βάση την τριγωνική ανισότητα.

```
lower_bound = i * maxd + dist_q_p[i] - epsilon
upper_bound = i * maxd + dist_q_p[i] + epsilon
```

Κάνοντας δυαδική αναζήτηση απευθείας στο `lower_bound` χρησιμοποιώντας τη συνάρτηση `bisect`, ο δείκτης αναζήτησης μεταφέρεται ακαριαία στο σωστό σημείο εκκίνησης, αποφεύγοντας την άσκοπη σειριακή σάρωση. Κατά την σάρωση των υποψηφίων αν ο αλγόριθμος ξεπεράσει το άνω όριο η σάρωση σταματάει και αξιοποιώντας τον πίνακα `closest_pivot_idx`, επαληθεύεται σε χρόνο $O(1)$ ότι το αντικείμενο όντως ανήκει στο τρέχον partition i και δεν προέκυψε από σφάλμα από την ακρίβεια της κινητής υποδιαστολής.

Μέρος 4: Ερωτήματα ομοιότητας kNN (kNN similarity queries)

Στο τέταρτο μέρος της εργασίας υλοποιήσαμε τα ερωτήματα ομοιότητας kNN similarity queries. Στόχος είναι για ένα σημείο query q και έναν θετικό ακέραιο k , να βρεθεί το σύνολο των k αντικειμένων o με τη μικρότερη $\text{dist}(q,o)$. Για τη διαχείριση των k κοντινότερων αντικειμένων χρησιμοποιήσαμε μια Ουρά Προτεραιότητας (Priority Queue) και συγκεκριμένα έναν Max-Heap (μεγιστοβάθμιο σωρό). Δεδομένου ότι η ενσωματωμένη βιβλιοθήκη `heapq` της Python υλοποιεί Min-Heap, αποθηκεύσαμε τις αποστάσεις με αρνητικό πρόσημο, ώστε το στοιχείο στην κορυφή του σωρού να αντιστοιχεί πάντα στη μέγιστη απόσταση της τρέχουσας k -άδας, η οποία λειτουργεί ως το τρέχον κατώφλι αναζήτησης ϵ .

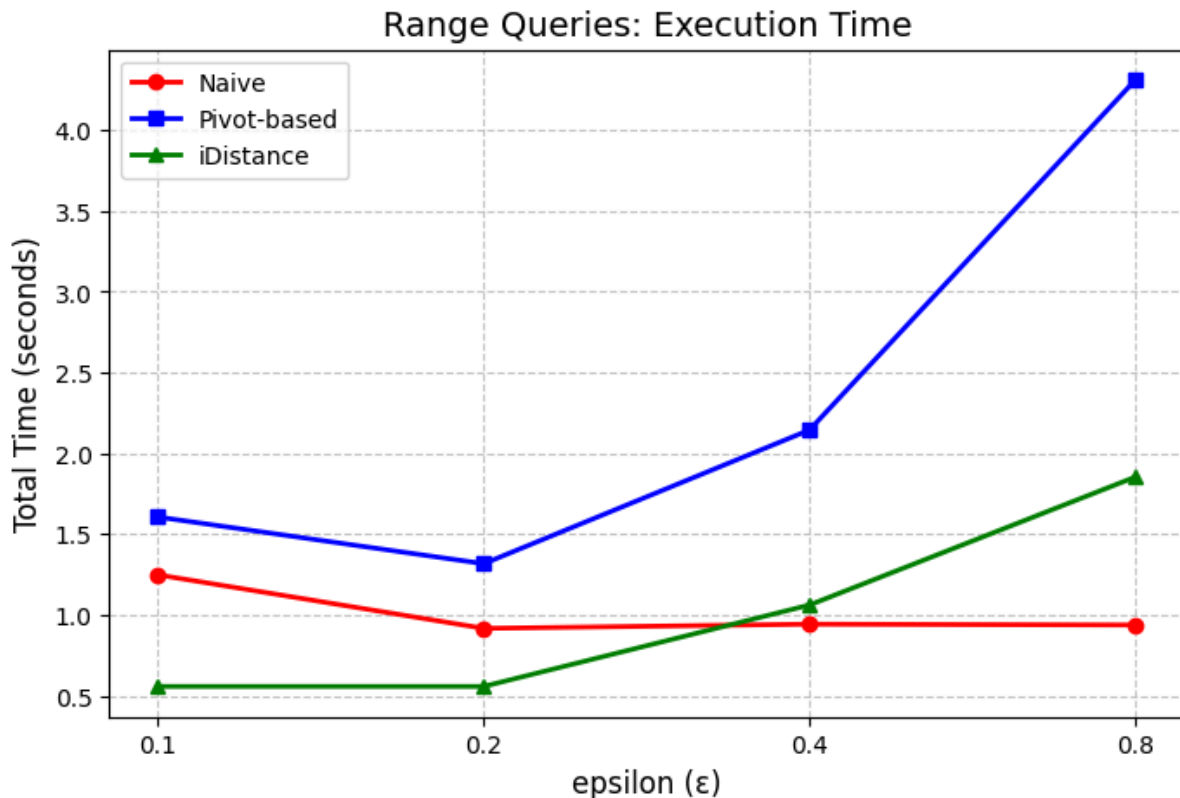
Για την **Naive kNN προσέγγιση** σαρώνουμε σειριακά όλα τα αντικείμενα και για κάθε αντικείμενο υπολογίζεται η απόστασή του από το q . Αν έχουμε λιγότερα από k αντικείμενα τότε το νέο αντικείμενο προστίθεται χωρίς περαιτέρω ελέγχους. Στην περίπτωση όμως που έχουμε περισσότερα από k αντικείμενα τότε ελέγχουμε την απόσταση του νέου αντικειμένου με την κορυφή του σωρού και αν είναι μικρότερη τότε το στοιχείο της κορυφής αφαιρείται και εισάγουμε το νέο αντικείμενο.

Για την **Pivot-based kNN προσέγγιση** συνδυάζουμε την τριγωνική ανισότητα με τον σωρό. Σε πρώτο στάδιο γεμίζουμε τον σωρό με τα πρώτα k αντικείμενα και στην συνέχεια ορίζουμε το τρέχον όριο ϵ με βάση το στοιχείο στην κορυφή. Μετά για τα επόμενα στοιχεία ελέγχεται η τριγωνική ανισότητα $|\text{dist}(p_i,o) - \text{dist}(p_i,q)| > \epsilon$ και για όποια στοιχεία ισχύει γίνεται pruning και δεν υπολογίζεται η απόσταση του αντικειμένου. Αλλιώς υπολογίζεται η απόσταση από το q και εφόσον είναι μικρότερη του ϵ τότε το νέο αντικείμενο εισάγεται στην σωρό δημιουργώντας μία νέα κορυφή και αλλάζεται το όριο ϵ κάνοντας τα κριτήρια απόρριψης ολοένα και πιο δύσκολο να επιτευχθούν.

Τέλος για την **iDistance-based kNN προσέγγιση** σε πρώτο στάδιο βρίσκουμε το αντικείμενο που είναι κοντινότερο στο σημείο ερώτησης q και υπολογίζουμε τα όρια του `partition closest_p * maxd` έως `(closest_p + 1) * maxd`. Στην συνέχεια χρησιμοποιούμε την δυαδική αναζήτηση `bisect` για να σαρώσουμε μόνο τα αντικείμενα αυτού του `partition` γεμίζοντας τον σωρό και διαμορφώνοντας ένα ικανοποιητικό αρχικό όριο ϵ . Αφού ολοκληρωθεί αυτή η πρώτη δουλειά συνεχίζουμε ελέγχοντας τα υπόλοιπα pivots. Αν έχει ολοκληρωθεί η συμπλήρωση k στοιχείων στον σωρό τότε το ϵ θα ισούται με την κορυφή του σωρού αλλιώς θα τίθεται στο άπειρο. Για κάθε καινούργιο pivot i θα ελέγχεται αν όλο το `partition` στο οποίο ανήκει μπορεί να απορριφθεί μαζικά ή όχι και αν δεν μπορεί να απορριφθεί τότε υπολογίζονται τα βασικά όρια αναζήτησης $[i * \text{maxd} + \text{dist_q_p}[i] - \epsilon, i * \text{maxd} + \text{dist_q_p}[i] + \epsilon]$ και βρίσκουμε την τομή τους με τα φυσικά όρια του `partition i`. Τέλος αφού βρεθεί η τομή ξεκινάμε με δυαδική αναζήτηση την σάρωση της τομής και κάθε φορά που ένα νέο αντικείμενο εισέρχεται στον σωρό το όριο ϵ θα μικραίνει. Αυτό μας κάνει να ανανεώσουμε και το άνω όριο που είχαμε αρχικά υπολογισμένο και να το “τραβήξουμε πιο πίσω” καθώς το ϵ μικραίνει σταματώντας την αναζήτηση από το να κάνει άσκοπους υπολογισμούς και επιτυγχάνοντας έτσι βέλτιστη ταχύτητα. Σε όλα αυτά χρησιμοποιείται και εδώ ο πίνακας `closest_pivot_idx` για την ασφαλή ταυτοποίηση των αντικειμένων εξαλείφοντας σφάλματα υποδιαστολής.

Μέρος 5: Αξιολόγηση

Στο πέμπτο και τελευταίο μέρος θα σας παρουσιάσουμε τα διαγράμματα που ζητήθηκαν καθώς και τις παρατηρήσεις και συμπεράσματα για αυτά. Ξεκινώντας με ερωτήματα ομοιότητας εύρους έχουμε τα εξής συμπεράσματα:

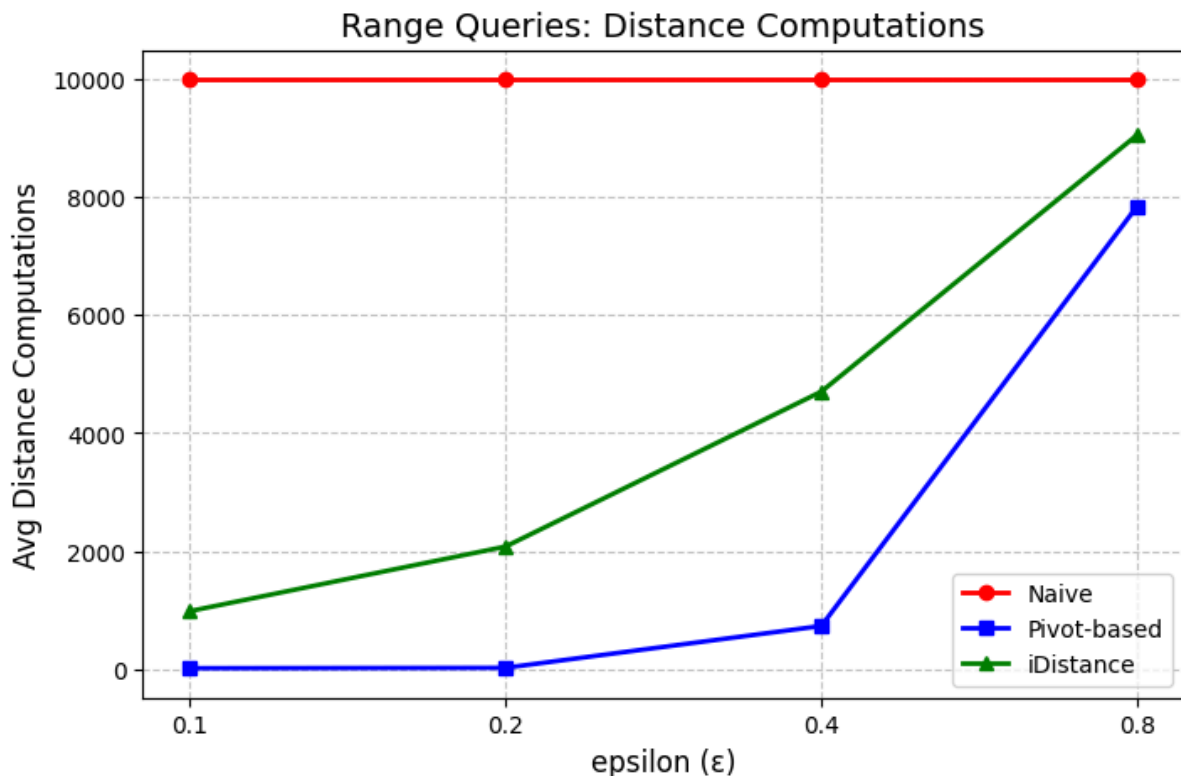


Στο πρώτο διάγραμμα βλέπουμε τον συνολικό χρόνο εκτέλεσης κάθε μεθόδου για τα ερωτήματα range queries.

Παρατηρούμε ότι η **iDistance** μέθοδος έχει μία καθαρή υπεροχή στους χρόνους για μικρά ϵ και γενικά χαμηλές ταχύτητες αποδεικνύοντας την υπεροχή στην ταχύτητα της δυαδικής αναζήτησης πάνω σε ταξινομημένο 1D πίνακα.

Παρατηρούμε επίσης μία σταθερά χαμηλή ταχύτητα για την **Naive** μέθοδο κοντά στον 1 δευτερόλεπτο το οποίο είναι απόλυτα λογικό καθώς κάνει πλήρη σειριακή σάρωση όλα τα δεδομένα εκτελώντας τον ίδιο όγκο εργασίας ανεξάρτητα από την ακτίνα ερωτήματος.

Τέλος η **Pivot-Based** μέθοδος αποδεικνύεται η πιο κοστοβόρα από άποψη χρόνου ιδίως για μεγαλύτερες τιμές του ϵ . Αυτό συμβαίνει καθώς για μεγαλύτερες τιμές του ϵ το κριτήριο της τριγωνικής ανισότητας γίνεται αρκετά δύσκολο στο να ικανοποιηθεί και άρα η μέθοδος δεν καταφέρνει να απορρίψει τόσο εύκολα αντικείμενα καταλήγοντας να κάνει διπλή δουλειά και άρα και παραπάνω χρόνο εκτέλεσης.



Στο δεύτερο αυτό διάγραμμα βλέπουμε τον μέσο αριθμό υπολογισμών απόστασης σε συνάρτηση με την ακτίνα αναζήτησης (ϵ).

Παρατηρούμε πολύ λογικά ότι η **Naive** μέθοδος έχει σταθερή τιμή τους 10 χιλιάδες υπολογισμούς και δεν κάνει κανένα φιλτράρισμα στα σημεία που υπολογίζεται η απόσταση, καθώς ο αλγόριθμος περνάει πάντα σταθερά από κάθε αντικείμενο του συνόλου. Παρατηρούμε επίσης ότι για τις άλλες δύο μεθόδους όσο αυξάνεται η τιμή ϵ αυξάνεται δραματικά ο αριθμός των υπολογισμών κάτι πολύ λογικό καθώς αυξάνεται και η ανάγκη τσεκαρίσματος της απόστασης σε περισσότερα αντικείμενα όσο πιο μεγάλο είναι το ϵ .

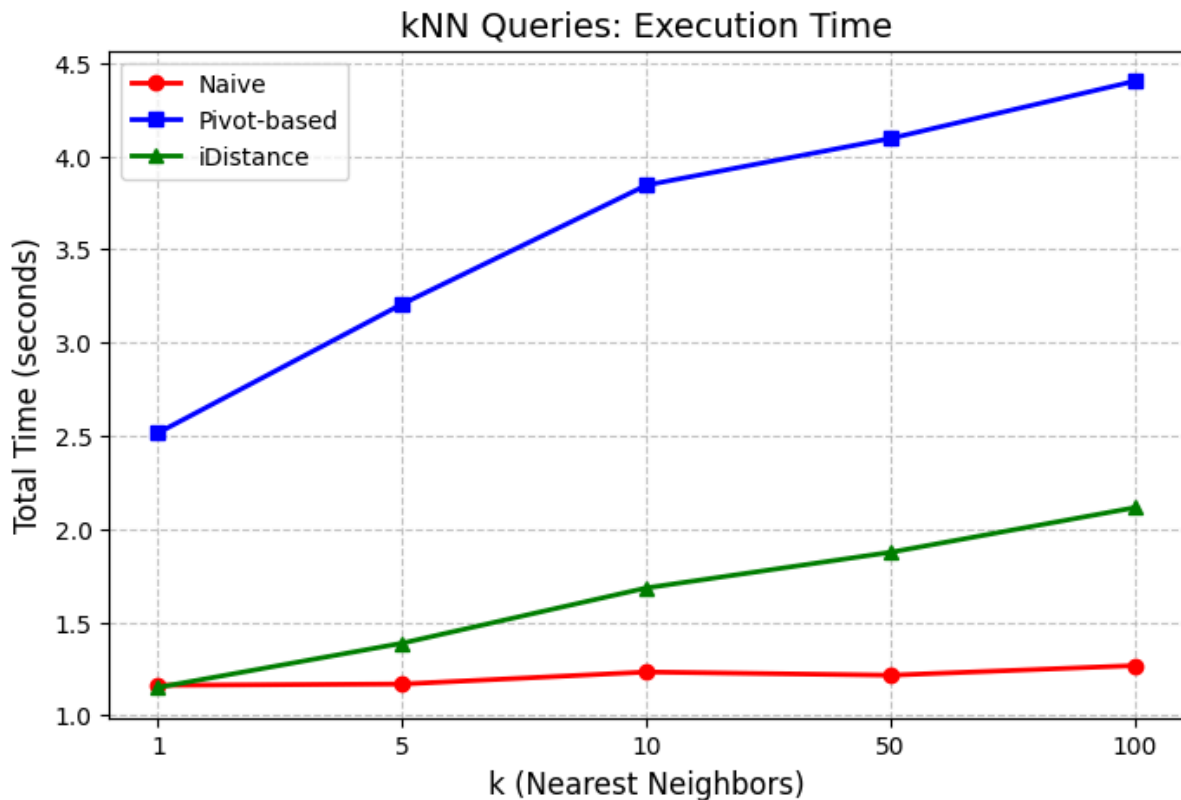
Για την **Pivot-Based** αυτή την φορά πάντως βλέπουμε ότι είναι η καλύτερη μέθοδος από την άποψη υπολογισμών αποστάσεων από τις άλλες ιδίως για μικρές τιμές αφού με βάση την τριγωνική ανισότητα μπορεί να απορρίπτει πολύ μεγάλα κομμάτια του συνόλου αντικειμένων πολύ γρήγορα. Βέβαια βλέπουμε και μία ραγδαία αύξηση για $\epsilon = 0.8$ αφού εκεί πλέον ακόμα και η τριγωνική ανισότητα δεν επαρκεί για να αποκλείσουμε αντικείμενα πλέον.

Τέλος η **iDistance** παρουσιάζει μία σταθερά αυξητική τάση με υπολογισμούς αποστάσεων πάντα περισσότερους από την Pivot-Based κάτι απόλυτα λογικό αν αναλογιστούμε το γεγονός ότι συμπιέζουμε πληροφορίες 10 διαστάσεων σε 1 διάσταση. Αυτό σημαίνει πως έχουμε μεγάλη απώλεια γεωμετρική πληροφορίας η οποία είναι πολύ απαραίτητη για να μειώνουμε τους απαραίτητους υπολογισμούς αποστάσεων.

Συμπερασματικά για τα Range Queries μπορούμε να πούμε με βάση αυτά τα πειράματα ότι η μέθοδος iDistance είναι η βέλτιστη επιλογή ιδίως για μεγάλα σύνολα

πολυδιάστατων δεδομένων καθώς έχει τους μικρότερους χρόνους εκτέλεσης. Από την άποψη ελαχιστοποίησης υπολογισμών όμως νικάει η Pivot-based μέθοδος. Η χειρότερη μέθοδος είναι σε τελική ανάλυση η Naive με την κύρια χρησιμότητα της να είναι να λειτουργεί ως baseline για να ελέγχουμε ορθότητα αποτελεσμάτων.

Πάμε τώρα να αναλύσουμε και τα αποτελέσματα για τα ερωτήματα ομοιότητας kNN. Τα συμπεράσματα που εξαγάμε είναι τα εξής:



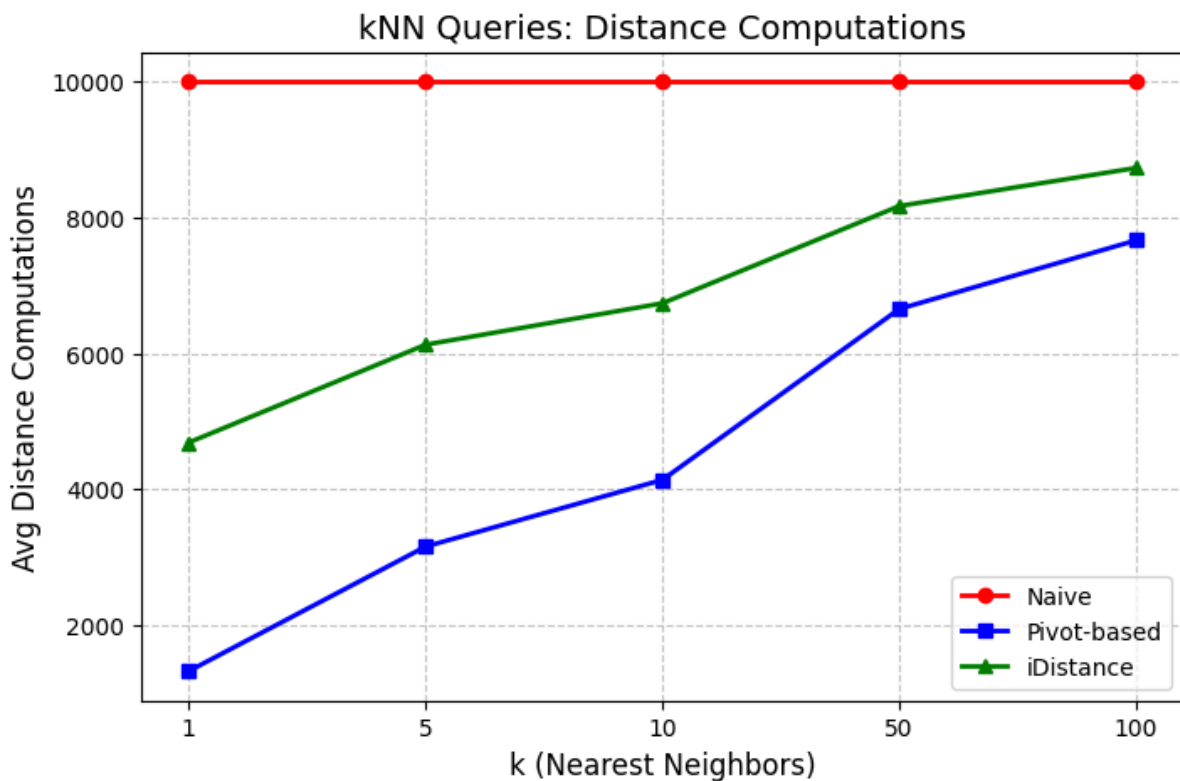
Εδώ βλέπουμε σε διάγραμμα τον συνολικό χρόνο εκτέλεσης κάθε μεθόδου σε συνάρτηση με την τιμή των k κοντινότερων γειτόνων.

Αρχικά να πούμε πως τους μικρότερους και πάντα σταθερούς χρόνους εκτέλεσης τους έχει σταθερά η **Naive** όπως βλέπουμε και στο διάγραμμα αφού όπως και πριν υπολογίζει υποχρεωτικά την απόσταση για όλα τα 10000 αντικείμενα του συνόλου. Αυτό δεν σημαίνει αναγκαστικά ότι η Naive θα είναι πάντα καλύτερη καθώς αν είχαμε μεγαλύτερα dataset θα είχε σταθερά πολύ μεγαλύτερους χρόνους από τις άλλες μεθόδους. Στο συγκεκριμένο μικρό dataset όμως η Naive είναι η καλύτερη σε άποψη χρόνου.

Όσον αφορά την **Pivot-based** μέθοδο παρατηρούμε ότι έχει τους χειρότερους χρόνους ακόμα και για $k = 1$ που απαιτεί σχεδόν διπλάσιο χρόνο από τις άλλες δύο. Καθώς το k μεγαλώνει το όριο φιλτραρίσματος γίνεται πολύ αργά αυστηρό και ως αποτέλεσμα να κάνουμε διπλούς υπολογισμούς (pruning attempts) καθώς και τους αυξημένους υπολογισμούς απόστασης.

Νικητής στους χρόνους μοιάζει να είναι η **iDistance** μέθοδος που παρουσιάζει

απλά μια σταθερή αλλά λογική αύξηση στους χρόνους σε συνάρτηση με την αύξηση στον αριθμό των γειτόνων που αναζητούμε. Αυτό είναι απόλυτα αναμενόμενο καθώς για περισσότερους γείτονες η ακτίνα αναζήτησης διατηρείται για μεγαλύτερο χρόνο και εν συνέχεια σαρώνονται περισσότερα partitions.



Σε αυτό το τελευταίο διάγραμμα βλέπουμε τον μέσο αριθμό υπολογισμών απόστασης σε σχέση με το πλήθος των k κοντινότερων γειτόνων.

Όπως και στο προηγούμενο διάγραμμα για τα ερωτήματα ομοιότητας εύρους η **Naive** μέθοδος κάνει τους περισσότερους υπολογισμούς με σταθερή τιμή το 10000 το οποίο είναι απόλυτα λογικό καθώς ελέγχει την απόσταση των σημείων όλου του dataset.

Εδώ και πάλι βλέπουμε την υπεροχή στους υπολογισμούς της **Pivot-based** μεθόδου. Όπως και πριν παραμένει η πιο αποτελεσματική μέθοδος ως προς την μαθηματική απόρριψη (pruning). Παρόλα αυτά όσο το k μεγαλώνει αυξάνονται και οι απαραίτητοι υπολογισμοί αφού ο σωρός (heap) χρειάζεται περισσότερα αντικείμενα για να γεμίσει και ως αποτέλεσμα η δυναμική ακτίνα παραμένει μεγαλύτερη για περισσότερο χρόνο.

Τέλος όπως και πριν η **iDistance** μέθοδος εκτελεί σταθερά περισσότερους υπολογισμούς από την Pivot-based. Όπως και πριν ο λόγος είναι καθαρά η συμπίεση των 10 διαστάσεων σε 1, όπου λόγω της απώλειας γεωμετρικής πληροφορίας ο αλγόριθμος βλέπει περισσότερα υποψήφια αντικείμενα που στις 10 διαστάσεις θα ήταν πιο μακριά.

Κάνοντας μία τελική ανάλυση καταλαβαίνουμε πως παρόλο που η Naive μέθοδος φαίνεται να είναι η πιο γρήγορη για τα kNN queries αυτό οφείλεται καθαρά στο μικρό dataset που έχουμε. Σε μεγαλύτερα dataset η naive μέθοδος μπορεί να έκανε μέχρι και μερικά λεπτά να ολοκληρωθεί καθώς δεν είναι κλιμακώσιμη όπως οι άλλες δύο μέθοδοι. Άρα η βέλτιστη και κλιμακώσιμη λύση είναι η iDistance καθώς πετυχαίνει τους μικρότερους δυνατούς χρόνους ιδίως για μεγάλα dataset.

Οδηγίες εκτέλεσης

Ο κώδικας είναι υλοποιημένος σε Python και γράφτηκε σε ένα αρχείο python με όνομα `vector_search.py`. Πριν την εκτέλεση, βεβαιωθείτε ότι όλα τα απαραίτητα αρχεία βρίσκονται στον ίδιο φάκελο (directory). Συγκεκριμένα:

1. Το αρχείο του κύριου κώδικα (`vector_search.py`).
2. Το αρχείο δεδομένων (`data10K10.txt`).
3. Το αρχείο των ερωτημάτων (`queries10.txt`).

Για την εκτέλεση του προγράμματος εκτελείτε την παρακάτω εντολή στο τερματικό:

`python vector_search.py <αριθμός pivots> <ακτίνα αναζήτησης> <Πλήθος γειτόνων>`

π.χ `python vector_search.py 10 0.2 5`